

学号： 2104230414

沈阳建筑大学

上机报告

课程名称： 算法设计与分析

项目名称： 动态规划(二)

2023年秋季学期

上机时间：第 3 周 星期 二 第 3 ~ 4 节

学生姓名： 张清晨 专业班级： 计算机 2101

指导教师： 刘俊岭 成 绩：

项目名称	动态规划(二)		学时	4
日期：2023 年 9 月 12 日		第 3 教学周		
硬件及软件				
名 称		版本、型号		
Visual C++		6.0 及以上		
上机目的与要求： 1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度与空间复杂度。 本次任务： 实验 3-3：01 背包问题实验 实验 3-4：完全背包 实验 3-5：最长公共子序列				

实验 3-3: 01 背包问题实验

1. 问题描述, 题目定义。

有 n 个物品, 每个物品有重量 $weight$ 和价值 $profit$ 两个属性, 还有一个承重为 c 的背包。现在让你用这个背包装物品, 使所装物品的价值最大化。

2. 算法实例, 实例演示程序过程。

■ 最优解追踪

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j - weight[i]] + profit[i]);$$

■ 记录决策过程

$rec[i][j]=1$, 选择商品

0, 不选商品

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

capacity -->									
index	0	1	2	3	4	5	6	7	
0	0	1	1	1	1	1	1	1	
1	0								$rec[i][j]=0$
2	0								
3	0								

Diagram illustrating the DP table and decision recording process. The table shows the maximum profit $dp[i][j]$ for items 0 to 3 and capacities 0 to 7. The decision recording array $rec[i][j]$ is shown with red boxes indicating where the decision was made (1 for "select item", 0 for "do not select item").

- For item 1, $rec[1][2]=1$ (select item 1) and $rec[1][5]=0$ (do not select item 1).
- For item 2, $rec[2][3]=1$ (select item 2) and $rec[2][5]=1$ (select item 2).

3. 程序代码, 完整可运行程序, 并加注释。

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>
```

```
using namespace std;
```

```
void Knapsack(int profit[], int weight[], int c, int n, int **dp, int **rec);
```

```

void Traceback(int **dp, int weight[], int c, int n, int *x, int **rec);

int main(){
    int profit[5] = { 0, 1, 6, 10, 16 };//第一个数没用
    int weight[5] = { 0, 1, 2, 3, 5 };//第一个数没用
    int c = 7, n = 4;

    int ** dp = new int*[n + 1]; //共有 length1+1 行
    int ** rec = new int*[n + 1];
    for (int i = 0; i < n + 1; i++)
    {
        dp[i] = new int[c + 1]; //共有 length2+1 列
        rec[i] = new int[c + 1];
    }
    for (int j = 0; j < n + 1; j++)
        dp[0][j] = 0;
    int *x=new int[c + 1];

    int sumWeight=0, sumProfit=0;

    Knapsack(profit, weight, c, n, dp, rec);
    Traceback(dp, weight, c, n, x, rec);
    printf("放入背包的物品为: \n");
    for (int i = 1; i < c ; i++)
    {

        if (x[i] == 1){
            printf("第%d 个, 重量为: %d, 价值为: %d\n", i, weight[i], profit[i]);
            sumWeight += weight[i] ;
            sumProfit += profit[i] ;
        }
        //printf("%d ", x[i]);
    }
    printf("总重量为: %d, 总价值为: %d\n", sumWeight, sumProfit);

    //打印 dp 数组和 rec 数组
    printf("dp 数组为: \n");
    for (int i = 1; i <= n; i++)
    {
        for (int j = 0; j <= c; j++)
        {
            printf("dp[%d][%d]=%d\t", i, j, dp[i][j]);
        }
        printf("\n");
    }
    printf("rec 数组为: \n");
    for (int i = 1; i <= n; i++)

```

```

{
    for (int j = 0; j <= c; j++)
    {
        printf("rec[%d][%d]=%d\t", i, j, rec[i][j]);
    }
    printf("\n");
}

system("PAUSE");
return 0;
}

void Knapsack(int profit[], int weight[], int c, int n, int **dp, int **rec)
{
    int jMax = min(weight[1] - 1, c); //背包剩余容量上限 范围[0~w[n]-1]
    for (int j = 0; j <= jMax; j++)
    {
        dp[1][j] = 0; //第一行初始化
        rec[1][j] = 0;
    }

    for (int j = weight[1]; j <= c; j++) //限制范围[w[n]~c]
    {
        dp[1][j] = profit[1]; //***
        rec[1][j] = 1;
    }

    for (int i = 2; i <= n; i++)
    {
        jMax = min(weight[i] - 1, c);
        for (int j = 0; j <= jMax; j++) //背包不同剩余容量 j<=jMax<c
        {
            dp[i][j] = dp[i - 1][j]; //没产生任何效益
            rec[i][j] = 0;
        }

        for (int j = weight[i]; j <= c; j++) //背包不同剩余容量 j-wi > c
        {
            dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + profit[i]); //效益值增长
vi
            if (dp[i - 1][j] >= dp[i - 1][j - weight[i]] + profit[i])
                rec[i][j] = 0;
            else rec[i][j] = 1;
        }
    }
}
}

```

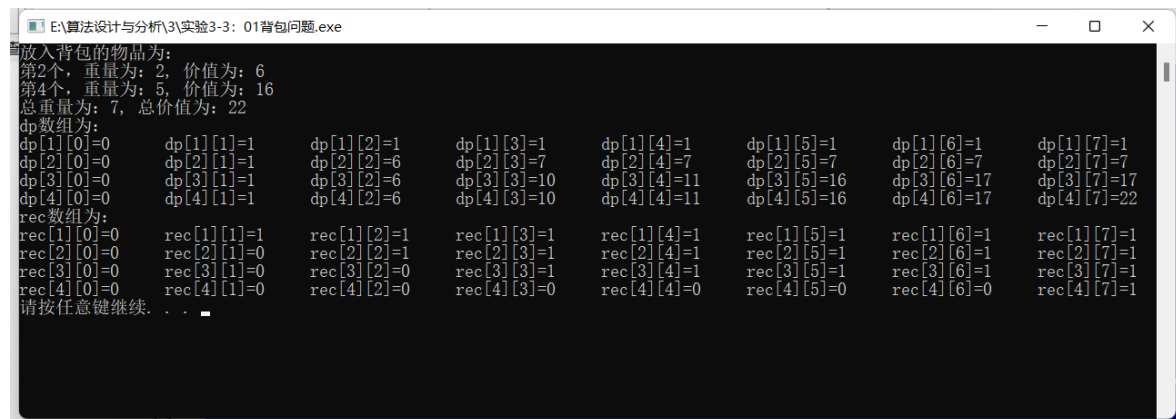
//x[]数组存储对应物品 0-1 向量,0 不装入背包, 1 表示装入背包

```
void Traceback(int **dp, int weight[], int c, int n, int *x, int **rec)
```

```
{
    for (int i = n; i >= 1; i--)
    {
        if (dp[i][c] == dp[i - 1][c])
        {
            x[i] = 0 ;
        }
        else
        {
            x[i] = 1 ;
            c -= weight[i];
        }
    }
    //x[n] = (dp[n][c]) ? 1 : 0;
}
```

4. 程序验证, 要求有不同的输入、输出, 并截图。

```
int profit[5] = { 0, 1, 6, 10, 16 }; // 第一个数没用
int weight[5] = { 0, 1, 2, 3, 5 }; // 第一个数没用
int c = 7, n = 4;
```



```
int profit[5] = {0, 1, 9, 10, 16};
int weight[5] = {0, 1, 5, 2, 3};
int c = 7, n = 4;
```

```
E:\算法设计与分析\3\实验3-3: 01背包问题.exe
放入背包的物品为:
第1个, 重量为: 1, 价值为: 1
第3个, 重量为: 2, 价值为: 10
第4个, 重量为: 3, 价值为: 16
总重量为: 6, 总价值为: 27
dp数组为:
dp[1][0]=0    dp[1][1]=1    dp[1][2]=1    dp[1][3]=1    dp[1][4]=1    dp[1][5]=1    dp[1][6]=1    dp[1][7]=1
dp[2][0]=0    dp[2][1]=1    dp[2][2]=1    dp[2][3]=1    dp[2][4]=1    dp[2][5]=9    dp[2][6]=10    dp[2][7]=10
dp[3][0]=0    dp[3][1]=1    dp[3][2]=10    dp[3][3]=11    dp[3][4]=11    dp[3][5]=11    dp[3][6]=11    dp[3][7]=19
dp[4][0]=0    dp[4][1]=1    dp[4][2]=10    dp[4][3]=16    dp[4][4]=17    dp[4][5]=26    dp[4][6]=27    dp[4][7]=27
rec数组为:
rec[1][0]=0    rec[1][1]=1    rec[1][2]=1    rec[1][3]=1    rec[1][4]=1    rec[1][5]=1    rec[1][6]=1    rec[1][7]=1
rec[2][0]=0    rec[2][1]=0    rec[2][2]=0    rec[2][3]=0    rec[2][4]=0    rec[2][5]=1    rec[2][6]=1    rec[2][7]=1
rec[3][0]=0    rec[3][1]=0    rec[3][2]=1    rec[3][3]=1    rec[3][4]=1    rec[3][5]=1    rec[3][6]=1    rec[3][7]=1
rec[4][0]=0    rec[4][1]=0    rec[4][2]=0    rec[4][3]=1    rec[4][4]=1    rec[4][5]=1    rec[4][6]=1    rec[4][7]=1
请按任意键继续. . .
```

5. 算法分析，时间复杂度与空间复杂度。

该算法使用了动态规划，是动态规划类算法。打印 dp 数组和 rec 数组，空间复杂度为 $n*c$ 。如果是设置为滚动数组，空间复杂度为 n 。

时间复杂度: $O(n*c)$

空间复杂度: $O(n*c)$

实验 3-4：完全背包

1. 问题描述，题目定义。

有 n 件物品和一个最多能背重量为 c 的背包。第 i 件物品的重量是 $weight[i]$ ，得到的价值是 $profit[i]$ 。每件物品都有无限个（也就是可以放入背包多次），求解将哪些物品装入背包里物品价值总和最大。

2. 算法实例，实例演示程序过程。

3.3 背包问题 完全背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

• 实例

```
// 先遍历物品，再遍历背包
for (int i = 0; i < weight.size(); i++) { // 遍历物品
    for (int j = 1; j <= c; j++) { // 遍历背包容量
        if (j < weight[i])    dp[j] = dp[j];
        else    dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
    }
}
```

	j=0	1	2	3	4	5	6	7
i=0	0	1	2	3	4	5	6	7
1	0	1	6	7	12	13	18	19
2	0	1	6	10	12	?		
3								22

3. 程序代码，完整可运行程序，并加注释。

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

void completeKnapsack();

int main() {
    completeKnapsack();
    system("PAUSE");
}
```



```

    return 0;
}

void completeKnapsack()
{
    vector<int> weight = { 1, 2, 3, 5 };
    vector<int> profit = { 1, 6, 10, 16 };
    int c = 7;//背包容量
    vector<int> dp(c + 1, 0);
    vector<int> rec(c + 1, 0);
    for (int i = 0; i < weight.size(); i++) { // 遍历物品
        cout << "i=" << i << " ";
        for (int j = 1; j <= c; j++) { // 遍历背包容量
            if (j < weight[i])
            {
                dp[j] = dp[j];
                rec[j] = 0;
            }
            else
            {
                //dp[j] = max(dp[j], dp[j - weight[i]] + profit[i]);
                if (dp[j] > dp[j - weight[i]] + profit[i])
                {
                    dp[j] = dp[j];
                    rec[j] = 0;
                }
                else
                {
                    dp[j] = dp[j - weight[i]] + profit[i];
                    rec[j] = 1;
                }
            }

            cout << "dp[" << j << "]=" << dp[j] << " rec[" << j << "]=" << rec[j] << "\t";
            //cout << "j=" << j << " rec[j]=" << rec[j] << "\t";
        }
        cout << endl;
    }
    cout << "最大价值总和: " << endl;
    cout << dp[c] << endl;
}

```

4. 程序验证，要求有不同的输入、输出，并截图。

```
vector<int> weight = { 1, 2, 3, 5 };  
vector<int> profit = { 1, 6, 10, 16 };  
int c = 7; // 背包容量
```

```
i=0 dp[1]=1 rec[1]=1 dp[2]=2 rec[2]=1 dp[3]=3 rec[3]=1 dp[4]=4 rec[4]=1 dp[5]=5 rec[5]=1 dp[6]=6 rec[6]=1 dp[7]=7 rec[7]=1  
i=1 dp[1]=1 rec[1]=0 dp[2]=6 rec[2]=1 dp[3]=7 rec[3]=1 dp[4]=12 rec[4]=1 dp[5]=13 rec[5]=1 dp[6]=18 rec[6]=1 dp[7]=19 rec[7]=1  
i=2 dp[1]=1 rec[1]=0 dp[2]=6 rec[2]=0 dp[3]=10 rec[3]=1 dp[4]=12 rec[4]=0 dp[5]=16 rec[5]=1 dp[6]=20 rec[6]=1 dp[7]=22 rec[7]=1  
i=3 dp[1]=1 rec[1]=0 dp[2]=6 rec[2]=0 dp[3]=10 rec[3]=0 dp[4]=12 rec[4]=0 dp[5]=16 rec[5]=1 dp[6]=20 rec[6]=0 dp[7]=22 rec[7]=1  
最大价值总和:  
22  
请按任意键继续. . .
```

```
vector<int> weight = {1, 2, 3, 4};  
vector<int> profit = {1, 9, 6, 16};  
int c = 6;
```

```
i=0 dp[1]=1 rec[1]=1 dp[2]=2 rec[2]=1 dp[3]=3 rec[3]=1 dp[4]=4 rec[4]=1 dp[5]=5 rec[5]=1 dp[6]=6 rec[6]=1  
i=1 dp[1]=1 rec[1]=0 dp[2]=9 rec[2]=1 dp[3]=10 rec[3]=1 dp[4]=18 rec[4]=1 dp[5]=19 rec[5]=1 dp[6]=27 rec[6]=1  
i=2 dp[1]=1 rec[1]=0 dp[2]=9 rec[2]=0 dp[3]=10 rec[3]=0 dp[4]=18 rec[4]=0 dp[5]=19 rec[5]=0 dp[6]=27 rec[6]=0  
i=3 dp[1]=1 rec[1]=0 dp[2]=9 rec[2]=0 dp[3]=10 rec[3]=0 dp[4]=18 rec[4]=0 dp[5]=19 rec[5]=0 dp[6]=27 rec[6]=0  
最大价值总和:  
27  
请按任意键继续. . .
```

5. 算法分析，时间复杂度与空间复杂度。

本题使用动态规划算法，使用的是滚动数组，所以空间复杂度是物品数量 c ，时间复杂度是 $n * c$

- 时间复杂度: $O(n \times c)$
- 空间复杂度: $O(c)$

实验 3-5：最长公共子序列

1. 问题描述，题目定义。

给定两个序列 X 和 Y，求两个序列最长公共子序列。

2. 算法实例，实例演示程序过程。

	1	2	3	4	5	6	7
x_i	A	B	C	B	D	A	B
y_j	B	D	C	A	B	A	

c[]

	j=0	1	2	3	4	5	6
i=0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1
2	0	1	1	1	1	2	2
3	0	1	1	2	2	2	2
4	0	1	1	2	2	3	3
5	0	1	2	2	2	3	3
6	0	1	2	2	3	3	4
7	0	1	2	2	3	4	4

rec[]

	j=1	2	3	4	5	6
i=1	U	U	U	LU	L	LU
2	LU	L	L	U	LU	L
3	U	U	LU	L	U	U
4	LU	U	U	U	LU	L
5	U	LU	U	U	U	U
6	U	U	U	LU	U	LU
7	LU	U	U	U	LU	U

3. 程序代码，完整可运行程序，并加注释。

```
/**
 * 最长公共子序列。
 */
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>
#include <cstring>

using namespace std;

int LCSLength(char* str1, char* str2, int** rec);
void PrintLCS(int** rec, char* str1, int i, int j);

int main()
{
```

```

//双指针的方法申请动态二维数组
char str1[50]="ABCBADAB";
char str2[50]="BDCABA";

int length1 = strlen(str1);
int length2 = strlen(str2);

int** rec = new int*[length1 + 1]; //共有 length1+1 行
for (int i = 0; i < length1 + 1; i++)
    rec[i] = new int[length2 + 1]; //共有 length2+1 列

printf("公共最长子序列长度为: %d\n", LCSLength(str1, str2, rec));
printf("追踪为: \n");

PrintLCS(rec, str1, length1, length2);

printf("\n");
for (int i = 0; i < length1 + 1; i++) //释放动态申请的二维数组
    delete[] rec[i];
delete[] rec;

system("PAUSE");
return 0;
}

int LCSLength(char* str1, char* str2, int** rec)
{
    int i, j, length1, length2, len;
    length1 = strlen(str1);
    length2 = strlen(str2);

    //双指针的方法申请动态二维数组
    int** c = new int*[length1 + 1]; //共有 length1+1 行
    for (i = 0; i < length1 + 1; i++)
        c[i] = new int[length2 + 1]; //共有 length2+1 列

    for (i = 0; i < length1 + 1; i++)
        c[i][0] = 0; //第 0 列都初始化为 0
    for (j = 0; j < length2 + 1; j++)
        c[0][j] = 0; //第 0 行都初始化为 0

    for (i = 1; i < length1 + 1; i++)
    {
        for (j = 1; j < length2 + 1; j++)
        {
            if (str1[i - 1] == str2[j - 1]) //由于 c[i][j] 的 0 行 0 列没有使用, c[i][j] 的第 i 行元
                //素对应 str1 的第 i-1 个元素

```

```

        {
            c[i][j] = c[i - 1][j - 1] + 1;
            rec[i][j] = 0;    //搜索方向为左上 LU
        }
        else if (c[i - 1][j] > c[i][j - 1])
        {
            c[i][j] = c[i - 1][j];
            rec[i][j] = 1; //搜索方向为上 U
        }
        else
        {
            c[i][j] = c[i][j - 1];
            rec[i][j] = -1; //搜索方向为左 L
        }
    }
}
//打印 c[i][j]数组
printf("dp 数组: \n");
for (i = 0; i < length1 + 1; i++)
{
    for (int j = 0; j < length2 + 1; j++)
        printf("dp[%d][%d]=%d ", i, j, c[i][j]);
    printf("\n");
}
//打印 rec[i][j]数组
printf("rec 数组: \n");
for (i = 1; i < length1 + 1; i++)
{
    for (int j = 1; j < length2 + 1; j++)
        printf("rec[%d][%d]=%d ", i, j, rec[i][j]);
    printf("\n");
}

len = c[length1][length2];
for (i = 0; i < length1 + 1; i++)    //释放动态申请的二维数组
    delete[] c[i];
delete[] c;
return len;
}

void PrintLCS(int** rec, char* str1, int i, int j)
{
    if (i == 0 || j == 0)
        return;
    if (rec[i][j] == 0)
    {
        PrintLCS(rec, str1, i - 1, j - 1); //从后面开始递归，所以要先递归到子串的前面，
        然后从前往后开始输出子串
    }
}

```

```

        printf("%c", str1[i - 1]); //c[i][j]的第 i 行元素对应 str1 的第 i-1 个元素
    }
    else if (rec[i][j] == 1)
        PrintLCS(rec, str1, i - 1, j);
    else
        PrintLCS(rec, str1, i, j - 1);
}

```

4. 程序验证，要求有不同的输入、输出，并截图。

```

char str1[50]="ABCBDA";
char str2[50]="BDCABA";

```

```

E:\算法设计与分析\3\实验3-5: 最长公共子序列.exe
dp数组:
dp[0][0]=0 dp[0][1]=0 dp[0][2]=0 dp[0][3]=0 dp[0][4]=0 dp[0][5]=0 dp[0][6]=0
dp[1][0]=0 dp[1][1]=0 dp[1][2]=0 dp[1][3]=0 dp[1][4]=1 dp[1][5]=1 dp[1][6]=1
dp[2][0]=0 dp[2][1]=1 dp[2][2]=1 dp[2][3]=1 dp[2][4]=1 dp[2][5]=2 dp[2][6]=2
dp[3][0]=0 dp[3][1]=1 dp[3][2]=1 dp[3][3]=2 dp[3][4]=2 dp[3][5]=2 dp[3][6]=2
dp[4][0]=0 dp[4][1]=1 dp[4][2]=1 dp[4][3]=2 dp[4][4]=2 dp[4][5]=3 dp[4][6]=3
dp[5][0]=0 dp[5][1]=1 dp[5][2]=2 dp[5][3]=2 dp[5][4]=2 dp[5][5]=3 dp[5][6]=3
dp[6][0]=0 dp[6][1]=1 dp[6][2]=2 dp[6][3]=2 dp[6][4]=3 dp[6][5]=3 dp[6][6]=4
dp[7][0]=0 dp[7][1]=1 dp[7][2]=2 dp[7][3]=2 dp[7][4]=3 dp[7][5]=4 dp[7][6]=4
rec数组:
rec[1][1]=-1 rec[1][2]=-1 rec[1][3]=-1 rec[1][4]=0 rec[1][5]=-1 rec[1][6]=0
rec[2][1]=0 rec[2][2]=-1 rec[2][3]=-1 rec[2][4]=-1 rec[2][5]=0 rec[2][6]=-1
rec[3][1]=1 rec[3][2]=-1 rec[3][3]=0 rec[3][4]=-1 rec[3][5]=-1 rec[3][6]=-1
rec[4][1]=0 rec[4][2]=-1 rec[4][3]=1 rec[4][4]=-1 rec[4][5]=0 rec[4][6]=-1
rec[5][1]=1 rec[5][2]=0 rec[5][3]=-1 rec[5][4]=-1 rec[5][5]=1 rec[5][6]=-1
rec[6][1]=1 rec[6][2]=1 rec[6][3]=-1 rec[6][4]=0 rec[6][5]=-1 rec[6][6]=0
rec[7][1]=0 rec[7][2]=1 rec[7][3]=-1 rec[7][4]=1 rec[7][5]=0 rec[7][6]=-1
公共最长子序列长度为: 4
追踪为:
BDAB
请按任意键继续. . .

```

```

char str1[50] = "AABCD";
char str2[50] = "BBCCD";

```

```
选择 E:\算法设计与分析\3\实验3-5: 最长公共子序列.exe
dp数组:
dp[0][0]=0 dp[0][1]=0 dp[0][2]=0 dp[0][3]=0 dp[0][4]=0 dp[0][5]=0 dp[0][6]=0
dp[1][0]=0 dp[1][1]=0 dp[1][2]=0 dp[1][3]=0 dp[1][4]=0 dp[1][5]=0 dp[1][6]=0
dp[2][0]=0 dp[2][1]=0 dp[2][2]=0 dp[2][3]=0 dp[2][4]=0 dp[2][5]=0 dp[2][6]=0
dp[3][0]=0 dp[3][1]=1 dp[3][2]=1 dp[3][3]=1 dp[3][4]=1 dp[3][5]=1 dp[3][6]=1
dp[4][0]=0 dp[4][1]=1 dp[4][2]=1 dp[4][3]=2 dp[4][4]=2 dp[4][5]=2 dp[4][6]=2
dp[5][0]=0 dp[5][1]=1 dp[5][2]=1 dp[5][3]=2 dp[5][4]=2 dp[5][5]=3 dp[5][6]=3
dp[6][0]=0 dp[6][1]=1 dp[6][2]=1 dp[6][3]=2 dp[6][4]=2 dp[6][5]=3 dp[6][6]=4
rec数组:
rec[1][1]=-1 rec[1][2]=-1 rec[1][3]=-1 rec[1][4]=-1 rec[1][5]=-1 rec[1][6]=-1
rec[2][1]=-1 rec[2][2]=-1 rec[2][3]=-1 rec[2][4]=-1 rec[2][5]=-1 rec[2][6]=-1
rec[3][1]=0 rec[3][2]=0 rec[3][3]=-1 rec[3][4]=-1 rec[3][5]=-1 rec[3][6]=-1
rec[4][1]=1 rec[4][2]=-1 rec[4][3]=0 rec[4][4]=0 rec[4][5]=-1 rec[4][6]=-1
rec[5][1]=1 rec[5][2]=-1 rec[5][3]=1 rec[5][4]=-1 rec[5][5]=0 rec[5][6]=0
rec[6][1]=1 rec[6][2]=-1 rec[6][3]=1 rec[6][4]=-1 rec[6][5]=0 rec[6][6]=0
公共最长子序列长度为: 4
追踪为:
BCDD
请按任意键继续. . .
```

5. 算法分析，时间复杂度与空间复杂度。

时间复杂度为 $O(m*n)$

空间复杂度为 $O(m*n)$

结果分析和总结：

解决动态规划类问题，一般是有五个步骤：

•问题结构分析

给出问题表示，明确原始问题（明确 dp 数组的含义）

•递推关系建立

分析最优（子）结构，构造递推公式（推导递推公式）

•自底向上计算

确定计算顺序，依次求解问题（初始化、顺序遍历、打印 dp 数组）

•最优方案追踪

记录决策过程，输出最优方案

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：